MLA0301 – REINFORCEMENT LEARNING

ASSIGNMENT 1 – CO1

Programme	B.E. / B.Tech
Course Code / Name	MLA0301 – Reinforcement Learning
Course Outcome	CO1
Maximum Marks	50
Nature / Duration	Individual Assignment / 1 Week
Assignment Title	Design, Implement and Evaluate an Intelligent Reinforcement Learning Agent for Adaptive Smart Traffic Signal Management Using Markov Decision Processes and Bandit Algorithms
Submission Date	14/11/2026
Student Name	Karthik V
Register Number	192425071
GitHub Repository Link	MLA03_192425071_RL/Assignment 1 at main · KarthikMG-gf/MLA03_192425071_RL

Course Outcome (CO1): Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.

Bloom's Levels Assessed: K4 – Analyse and K5 – Evaluate

Project Scenario – Smart City Adaptive Traffic Signal Management

A smart city experiences changing vehicle density at four approaches of a road intersection. Fixed-time traffic signals cannot respond effectively to sudden congestion, leading to long waiting times, unnecessary idling and uneven traffic flow. The city administration proposes an intelligent traffic signal controller based on Reinforcement Learning (RL).

The controller observes the traffic condition, selects a signal action, receives a reward based on waiting time and queue length, and improves its decision-making strategy through interaction with the environment. The problem is modelled using an MDP, policies and value functions, Bellman equations, exploration versus exploitation, and a basic Bandit algorithm. TensorFlow/Keras are discussed as optional extensions for function approximation.

Design objective: Develop and justify an RL-based decision-making model that reduces vehicle waiting time and congestion while maintaining safe and fair signal operation.

Submission and execution notes

- Complete all five tasks individually and execute every program. Include source code, input data (if used), output evidence and explanations.
- Create a GitHub repository named MLA03-<RegisterNumber>-Assignment1 and share the link with the course faculty.
- Maintain separate folders named Task01, Task02, Task03, Task04 and Task05.
- Include a README.md with student name, register number, Python version, libraries, installation steps, execution commands and a brief result for each task.
- Commit work progressively, with at least one meaningful commit for each task.
- Show calculations clearly; state assumptions about traffic states, actions, rewards and episode termination.
- Do not claim that an experiment has been trained or evaluated unless it has actually been executed and its evidence retained.

TASK 1 – Analyse the Reinforcement Learning Framework and Model the Traffic Problem (8 Marks)

Question: Analyse the proposed smart traffic signal problem and formulate it as a Reinforcement Learning framework.

1.1 Five main elements of the RL framework

Element	Meaning in the traffic-signal problem
Agent	The intelligent traffic-signal controller. It observes traffic, selects a phase/timing action and updates its policy from rewards.
Environment	The four-approach road intersection, vehicles, queues, signal heads and traffic dynamics. It responds to the controller's actions.
State	A compact description of current traffic, such as the queue category on each approach and the currently active green phase.
Action	A legal signal-control decision, such as choosing the next approach to receive green or extending the current green for one

	time slot.
Reward	A scalar feedback value based on queue length, waiting time, throughput and safety/fairness penalties. Higher reward represents better traffic operation.

1.2 Suitable state representation and justification

Let the four approaches be North (N), East (E), South (S) and West (W). For each approach, classify the observed queue as Low (0–5 vehicles), Medium (6–15 vehicles) or High (more than 15 vehicles). Include the active signal phase as one of four values. The state is s = (qN, qE, qS, qW, phase). This is a compact, interpretable representation: queue categories describe congestion and phase is needed to enforce safe phase changes. The thresholds are modelling assumptions for this assignment and can be calibrated using real traffic data.

With four queue categories per approach? Here there are three categories per approach, so the queue portion has 3^4 = 81 combinations. Including four possible active phases gives 81 × 4 = 324 discrete states.

1.3 Actions and safety constraints

Action	Description
A0 – Keep current green	Continue the current legal phase for one additional control interval, subject to a maximum green limit.
A1 – Serve North/South	Select the compatible North–South movement after any required clearance interval.
A2 – Serve East/West	Select the compatible East–West movement after any required clearance interval.
A3 – Extend green	Extend the current green by one permitted time slot when the current queue justifies it.

Safety constraints: conflicting approaches must never receive green simultaneously; every phase change must include the prescribed yellow and all-red clearance intervals; minimum and maximum green times must be enforced; pedestrian crossing time must be respected where applicable; emergency-vehicle pre-emption must override normal learning; and a safe fixed-time fallback must be available if sensors or the controller fail. The learning agent chooses only among actions allowed by a safety mask.

1.4 Reward function

One suitable reward at decision time t is:

```
r_t = -0.5 * total_wait_t - 2.0 * total_queue_t + 3.0 * vehicles_cleared_t - 20.0 *
safety_violation_t - 1.0 * unfairness_t
```

Here total_wait is the sum of vehicle waiting time (or a normalized waiting measure) across approaches, total_queue is the total number of waiting vehicles, vehicles_cleared is the number that passed during the interval, safety_violation is 1 if a prohibited signal condition occurs and 0 otherwise, and unfairness penalizes repeatedly starving one approach. The numerical weights are illustrative design parameters and should be tuned using validation experiments. Positive rewards encourage throughput and good service; negative rewards discourage long queues, delay, unsafe actions and unfair service. A large safety penalty makes unsafe behaviour unacceptable, although safety must also be guaranteed by hard constraints rather than reward alone.

1.5 Learning interaction cycle

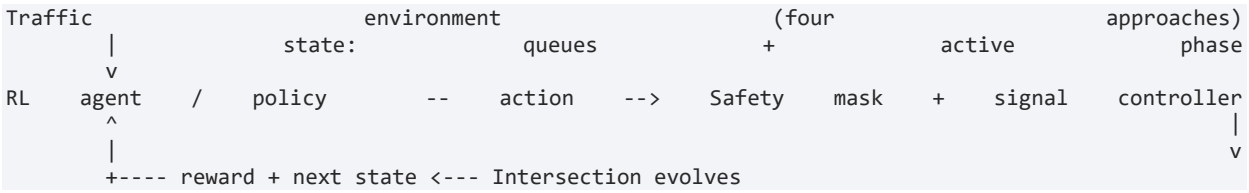
1. Observe the state s_t from queue sensors and the active phase.
2. Select a permitted action a_t using the current policy.
3. Apply the action, including required clearance timing.
4. The environment evolves and returns reward r_t and next state s_(t+1).
5. Update the policy or value estimate using (s_t, a_t, r_t, s_(t+1)).
6. Repeat until the simulated episode ends or continue operation in real time.

1.6 Assumptions; observability and task type

- Queue sensors provide sufficiently accurate queue categories at each decision interval.
- Vehicles obey the signal and the intersection geometry has known compatible movement phases.
- Arrivals and departures are stochastic; a simple simulator represents their variability.
- Control decisions occur at fixed intervals, and all phase timing and clearance constraints are known.
- An episode may represent a fixed simulated peak-hour period; deployment would be a continuing task because traffic control operates indefinitely.

The real intersection is not fully observable: exact driver intent, hidden queues, sensor errors and downstream blockages may not be visible. The proposed baseline treats the measured queue categories and active phase as a fully observed state approximation. If important information is missing, the problem is more accurately a partially observable MDP (POMDP), or the state can be augmented with recent observations.

1.7 RL framework diagram (text form)



1.8 Fixed-time control versus RL and complexity

Aspect	Fixed-time rule-based control	RL control
Decision rule	Pre-set cycle lengths and green splits, often repeated regardless of live queues.	Policy selects an action from the observed state and improves using reward feedback.
Adaptability	Low; changes require manual retuning or pre-programmed schedules.	Can adapt to changing demand represented in training/interaction data.
Data requirement	Can operate with timing plans and basic calibration; live sensing may be optional.	Needs state observations and reward feedback; training quality depends on representative traffic data/simulation.
Computation	Very small and predictable per decision.	Tabular action selection is O(|A|) per decision; a tabular update is O(1) for a selected state-action pair. A full Bellman sweep is O(|S|^2|A|) for a dense transition model, or O(|S||A||S'|) when summing over successors.
Main risk	Cannot react well to unexpected congestion.	May learn poorly outside training conditions; safety constraints and validation are essential.

For a fixed-time controller with a constant number of phases, choosing the next phase can be O(1). If it scans k candidate phases or timing rules, it is O(k). A tabular RL greedy action selection scans |A| actions, hence O(|A|).

TASK 2 – Formulate the Markov Decision Process (MDP) (10 Marks)

Question: Represent the smart traffic signal problem formally as a Markov Decision Process.

2.1 MDP tuple (S, A, P, R, γ)

Component	Definition in this problem
S	Finite set of traffic states, each describing queue categories and active phase.
A	Set of legal signal actions, such as keeping/ extending green or selecting a compatible phase.
P(s'|s,a)	Probability that traffic state s' follows current state s after action a, capturing random arrivals and departures.
R(s,a,s')	Immediate reward for the transition, based on delay, queue length, throughput, fairness and safety.
γ	Discount factor between 0 and 1, controlling the importance of future rewards.

2.2 Sample state set and action set

State	Traffic condition
S1	Light: all approaches have Low queues; North–South phase is active.
S2	Moderate: one or more approaches have Medium queues; North–South phase is active.
S3	Heavy East–West: East and West are High while North and South are Low/Medium; North–South phase is active.

Action	Meaning
A1	Continue/extend the current North–South green for one interval.
A2	After safe clearance, switch to East–West green.

2.3 Example transition-probability table

For the selected pair (S2, A1), assume the following illustrative transition probabilities. They represent a simple teaching model, not measured traffic data.

Current state	Action	Next state	Probability
S2	A1	S1 – Light	0.20

S2	A1	S2 – Moderate	0.60
S2	A1	S3 – Heavy E–W	0.20
		Total	1.00

All probabilities are between 0 and 1 and sum to 0.20 + 0.60 + 0.20 = 1.00, so the distribution is valid.

2.4 Rewards for next states

Next state	Illustrative reward	Justification
S1 – Light	+5	Queues are low and delay is small.
S2 – Moderate	+1	Traffic remains manageable but some waiting continues.
S3 – Heavy E–W	−4	The East–West queue is high and the selected action has not relieved it.

These values are illustrative. A practical reward should be calculated from measured waiting time, queue length, throughput and fairness rather than assigning a fixed score only by state.

2.5 Markov property

The Markov property states that, given the current state and action, the distribution of the next state is independent of the earlier history: P(S_(t+1)=s' | S_t=s, A_t=a, history) = P(s'|s,a). In traffic control, the current queue categories and active phase should summarize the information needed to predict the next queue state. If past arrival trends or time-of-day materially affect future arrivals and are not represented in the state, the state should be augmented with those features. Otherwise the Markov assumption may be inaccurate.

2.6 Role of discount factor γ

γ close to 0 makes the agent emphasize immediate reward. This can favour clearing the currently largest queue while ignoring future imbalance. γ close to 1 gives greater weight to future rewards and encourages decisions that improve long-term flow, but can make learning slower or more sensitive to uncertain long-term estimates. A value such as 0.9 is a reasonable illustrative starting point for a continuing traffic-control problem.

2.7 Numerical exercise

Given P(s1|s,a)=0.6, P(s2|s,a)=0.4, rewards 5 and −2, and γ=0.9, with next-state values equal to zero:

```
Expected  one-step  return  =  0.6  *  (5  +  0.9  *  0)  +  0.4  *  (-2  +  0.9  *  0)
                    =       0.6      *       5       +       0.4       *       (-2)
                    =                  3.0                  -                  0.8
                    = 2.2
```

Therefore, the expected one-step return is 2.2. If the next-state values are non-zero, include the discounted future value inside each transition contribution: G = Σ_s' P(s'|s,a) [R(s,a,s') + γV(s')]. The expected reward is then combined with the expected discounted future value.

2.8 Tabular MDP versus compact state representation and storage

A tabular representation stores explicit entries for states, actions, rewards and transitions. For a dense transition model, storage is O(|S|²|A|), because each state-action pair may transition to every state. If each pair has only d possible successors, sparse storage is O(|S||A|d). A compact feature representation stores only

relevant traffic features (queue levels, active phase, time slot or arrival-rate category) and computes or estimates values from those features. It reduces explicit storage when the full state space is large, but requires a model or function approximator. With m approaches and q queue categories per approach, queue combinations grow as q^m; adding features multiplies the number of combinations. State-space growth is therefore exponential in the number of independently discretized features.

TASK 3 – Analyse Policies, Value Functions and Bellman Equations (10 Marks)

Question: Study how an RL agent evaluates and improves its traffic signal decisions.

3.1 Policy π(a|s)

A policy π(a|s) gives the probability of choosing action a when the agent observes state s. A deterministic policy always chooses the same action for a given state; for example, if East–West is High and North–South is Low, it always switches to East–West after clearance. A stochastic policy assigns probabilities to actions; for example, it may choose East–West with probability 0.8 and continue North–South with probability 0.2, subject to safety constraints. Stochastic policies support exploration.

3.2 State-value and action-value functions

The state-value function Vπ(s) is the expected discounted sum of future rewards when the agent starts in state s and follows policy π. It tells the controller how favourable a traffic condition is under that policy. The action-value function Qπ(s,a) is the expected discounted return when the agent starts in state s, first takes action a, and then follows π. It allows the controller to compare candidate signal actions in the same state.

3.3 Bellman expectation equation

```
Vπ(s) = Σ_a π(a|s) Σ_s' P(s'|s,a) [ R(s,a,s') + γ Vπ(s') ]
```

π(a|s) is the probability of choosing action a in state s; P(s'|s,a) is the transition probability; R(s,a,s') is the immediate reward; γ is the discount factor; and Vπ(s') is the expected value of the next state under the same policy. The equation expresses the value of a state as the expected immediate reward plus discounted future value.

3.4 Bellman optimality equation

```
V*(s) = max_a Σ_s' P(s'|s,a) [ R(s,a,s') + γ V*(s') ]
```

V*(s) is the maximum achievable expected discounted return from state s. The maximization compares the expected return of each legal action. An optimal policy chooses an action attaining this maximum, while the safety mask removes prohibited actions.

3.5 Numerical action comparison

Given γ=0.8, Action A has immediate reward 4 and future value 8; Action B has immediate reward 6 and future value 5. The one-step action return is immediate reward plus γ times future value.

```
Action     A     =     4     +     0.8     *     8     =     4     +     6.4     =     10.4
Action     B     =     6     +     0.8     *     5     =     6     +     4.0     =     10.0
Preferred action: Action A, because 10.4 > 10.0.
```

Although Action B has the larger immediate reward, Action A has the larger total return after accounting for future value.

3.6 Why immediate-reward greed can be short-sighted

A greedy decision based only on immediate reward may serve the currently largest queue repeatedly and starve other approaches. This can create future congestion, longer total delay and unfair waiting. Bellman-based decisions consider both immediate reward and future state value, encouraging balanced long-term traffic flow.

3.7 Completed Bellman one-step value calculation code

```
gamma                                       =                                       0.8
probabilities                          =                          [0.6,                          0.4]
rewards                             =                             [5,                           -2]
next_state_values                             =                          [10,                           4]

def         bellman_return(probabilities,         rewards,         next_state_values,         gamma):
    total                                          =                                          0.0
    for                      i                      in                     range(len(probabilities)):
        contribution                =               probabilities[i]                *                (
            rewards[i]             +             gamma             *             next_state_values[i]
        )
        total                                      +=                                      contribution
    return                                                                                      total

print("Bellman                                return:",                                bellman_return(
    probabilities, rewards, next_state_values, gamma))
```

Expected output: Bellman return: 8.28

Calculation: $0.6 \times (5 + 0.8 \times 10) + 0.4 \times (-2 + 0.8 \times 4) = 0.6 \times 13 + 0.4 \times 1.2 = 7.8 + 0.48 = 8.28$. Therefore, the correct output is 8.28.

3.8 Repeated Bellman updates versus direct evaluation; complexity

For a small finite MDP with known transition probabilities and a fixed policy, direct policy evaluation solves a system of linear equations $(I - \gamma P\pi)V = R\pi$. It can be practical for small state spaces, but matrix solution cost is commonly $O(|S|^3)$ for dense matrices. Iterative policy evaluation repeatedly applies Bellman expectation updates until values change by less than a chosen tolerance. A dense full sweep over all states and actions and all possible next states costs $O(|S|^2|A|)$; with sparse transitions it costs $O(|S||A|d)$, where d is the average number of successors. Repeated sweeps are needed because the value estimate is propagated gradually through the state space until convergence. For $\gamma<1$ in a finite discounted MDP, iterative policy evaluation converges under standard conditions.

TASK 4 – Investigate Exploration, Exploitation and Bandit Algorithms (10 Marks)

Question: A traffic controller must decide which of four signal timing options to try. Some options may provide better traffic flow, but their actual rewards are initially unknown.

4.1 Exploration and exploitation dilemma

Exploration means trying different timing options to learn their reward characteristics, including options that currently appear weaker. Exploitation means selecting the option with the highest estimated reward so far. Too much exploration wastes opportunities to use a good timing plan; too much exploitation can lock the controller into a timing option that only appeared good because of limited early observations.

4.2 ε-greedy strategy

With probability ε, ε-greedy selects a random arm (exploration); with probability 1−ε, it selects the arm with the highest current estimated reward (exploitation). When ε=0, it is purely greedy and never deliberately explores. When ε=1, it always explores uniformly at random. Gradually reducing ε starts with more exploration and shifts toward exploitation as estimates become more reliable. A non-zero floor can preserve occasional exploration.

4.3 Upper Confidence Bound (UCB)

UCB selects the arm with the highest optimistic score, commonly:

```
UCB_i = Q_i + c * sqrt( ln(t) / N_i )
```

Q_i is the estimated mean reward for arm i, t is the total number of selections, N_i is the number of times arm i has been selected, and c controls the exploration strength. The uncertainty bonus is large for rarely selected arms and shrinks as more observations are collected. Each arm should be selected at least once before the formula is applied.

4.4 ε-greedy versus UCB

Aspect	ε-greedy	UCB
Decision strategy	Random exploration with probability ε; otherwise choose the current best estimate.	Choose the arm with the largest estimated reward plus uncertainty bonus.
Information used	Current reward estimates and ε.	Reward estimates, selection counts and total time step.
Advantage	Simple, inexpensive and easy to implement.	Targets uncertainty systematically and often explores under-sampled arms.
Limitation	Random exploration may repeatedly select clearly weak arms; fixed ε keeps exploring.	Requires count tracking and a suitable exploration coefficient; can be sensitive to non-stationary rewards.

4.5 Multi-armed bandit formulation

A multi-armed bandit has k alternative actions (arms), each producing an uncertain reward. In this experiment the four arms are four candidate signal timing options. The reward is a simulated score for traffic flow (or a negative delay/queue cost). The objective is to maximize cumulative reward or average reward over a fixed number of trials by learning which timing option performs best.

4.6 Why a Bandit is useful but incomplete for traffic control

A Bandit is suitable for comparing independent timing options when the same set of options can be tested and each trial produces a reward. However, it does not explicitly represent traffic states, the effect of a signal action on the next queue configuration, or long-term state transitions. It may therefore choose a timing plan that has a high average reward overall but performs poorly in a particular traffic condition. An MDP is needed to model state-dependent sequential decisions.

4.7 Completed ε-greedy multi-armed Bandit code

```
import                          numpy                          as                          np
```

```
np.random.seed(7)
true_rewards = [1.0, 1.5, 2.0, 1.2]  # illustrative reward means
n_arms = len(true_rewards)
epsilon = 0.1
steps = 1000

counts = np.zeros(n_arms, dtype=int)
estimates = np.zeros(n_arms, dtype=float)
total_reward = 0.0

for step in range(steps):
    if np.random.rand() < epsilon:
        action = np.random.randint(n_arms)  # explore
    else:
        action = int(np.argmax(estimates))  # exploit

    reward = np.random.normal(true_rewards[action], 1.0)

    counts[action] += 1
    estimates[action] += (reward - estimates[action]) / counts[action]
    total_reward += reward

print("Arm selections:", counts)
print("Estimated rewards:", np.round(estimates, 3))
print("Average reward:", round(total_reward / steps, 3))
```

The exact numerical output should be taken from executing the supplied script with NumPy and the stated seed. It is not appropriate to invent an output table. The separately supplied code package includes the executable script and a captured run output generated from execution.

4.8 Why the selected arm may differ from the highest true reward

A finite experiment uses a limited number of noisy reward samples. Random variation can make a lower-mean arm appear better temporarily, and exploration deliberately allocates some trials to non-best arms. Estimates approach the true means only with sufficient observations. A non-stationary traffic environment can also change the true reward over time.

4.9 ε-greedy versus purely greedy; complexity and memory

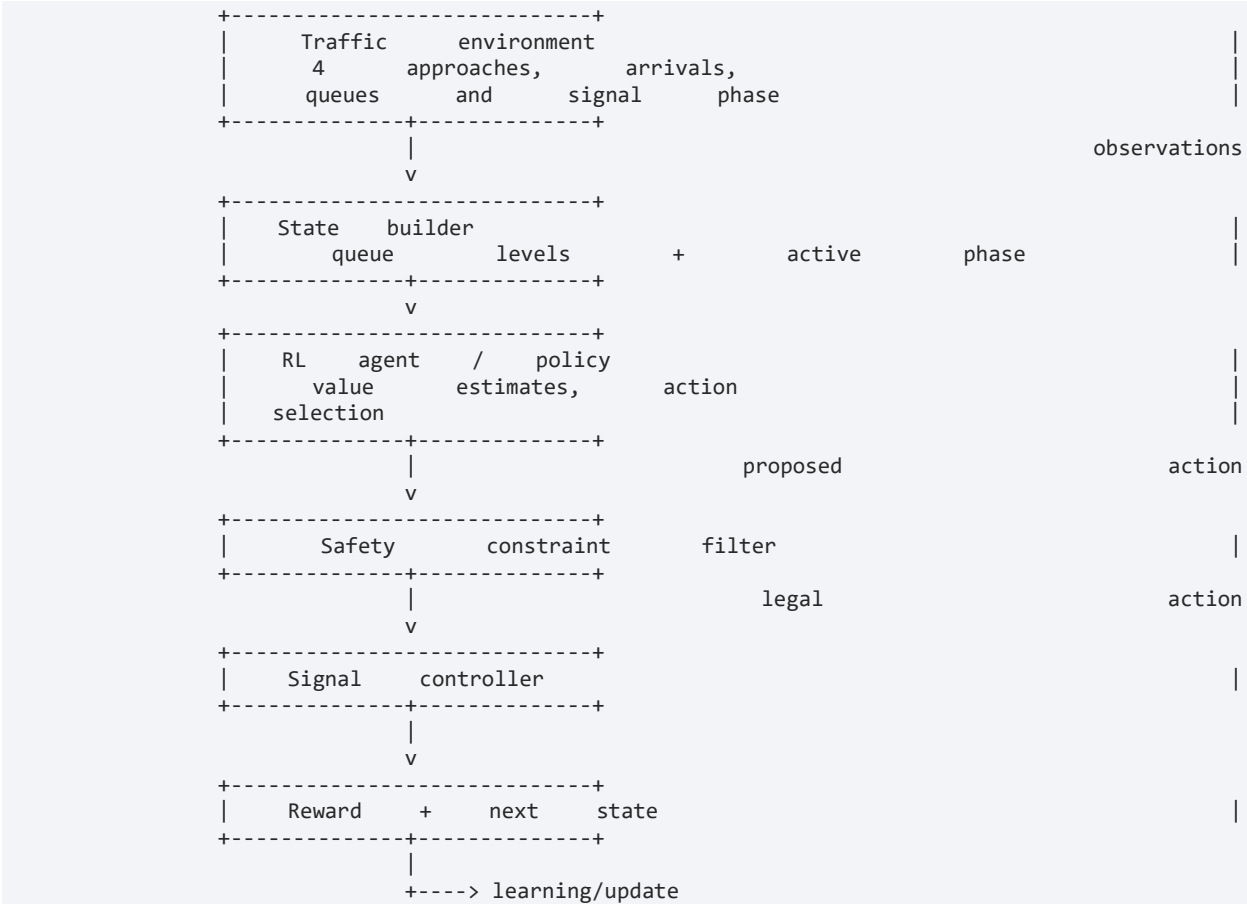
A purely greedy strategy always chooses the currently highest estimated arm and may never discover a better option if its initial estimate is poor. ε-greedy sacrifices some immediate reward for exploration and can correct early estimation errors. For k arms, finding the maximum estimate costs O(k) per action selection; updating one arm's count and incremental mean costs O(1). Storing one count and one estimate per arm costs O(k) memory.

TASK 5 – Develop, Evaluate and Defend the RL Project Design (12 Marks)

Question: Integrate the concepts from Tasks 1–4 and propose an implementable RL solution for the adaptive traffic signal problem.

5A. System design and workflow

Architecture: Traffic environment → state observation → safety filter → agent policy/action selection → signal execution → reward and next-state calculation → learning/update → next decision. The MDP defines the state, actions, transition model and reward. The policy selects actions. The value function estimates long-term outcomes. The Bandit experiment separately compares candidate timing options before or alongside the MDP controller.



Pseudocode for one interaction sequence

```
Initialize state representation, legal actions, reward weights and policy
Initialize value estimates and (if used) bandit counts/reward estimates
For each episode:
    Reset traffic queues and signal phase
    Observe current state s
    Until episode termination:
        Generate legal actions using signal safety constraints
        Select action a from policy π(a|s)
        Apply action and required yellow/all-red clearance
        Simulate vehicle arrivals and departures
        Observe next state s'
        Calculate reward r from delay, queue, throughput and fairness
        Update value/policy using (s, a, r, s')
        Set s = s'
    Record episode reward, average delay and queue statistics
Separately:
    Run fixed-time, greedy-bandit and ε-greedy-bandit comparisons
    Record reward, queue/waiting measure and action frequencies
```

5B. Python environment and library plan

Tool/library	Purpose
Python 3.10 or later	Main programming language for the simulator, agent, experiments and reproducible scripts.
NumPy	Arrays, random arrivals, probability calculations, reward estimates and numerical updates.
Matplotlib (optional)	Plots of rewards, queue lengths and action frequencies.

TensorFlow/Keras (optional extension)	Neural-network function approximation when a large or continuous state space makes a tabular value table impractical.
TensorFlow/Keras required for baseline?	No. The baseline in this assignment uses a small tabular/feature-based simulator and NumPy only.

Reproduction commands (example): python -m venv .venv; activate the environment; python -m pip install --upgrade pip; python -m pip install numpy matplotlib; then run the task scripts. Record the exact Python and library versions used on the submission machine in README.md.

5C. Experimental evaluation design

Compare three controllers under the same simulated traffic-arrival settings and random seed: (1) fixed-time baseline, which alternates North–South and East–West with a fixed green duration; (2) greedy Bandit, which always selects the timing arm with the highest current estimated reward; and (3) ε-greedy Bandit, which explores with probability ε and otherwise exploits. In this comparison, the Bandit arms are four candidate green-split/timing options. The experiment is a controlled timing-option comparison, not a full state-aware MDP controller.

Record at least: average reward per decision, mean total queue (as a waiting/congestion proxy), and selection frequency for each timing arm. For a stronger traffic study, also record average vehicle waiting time, throughput, maximum queue and per-approach fairness. Use the actual output generated by the experiment package; do not substitute expected or fabricated values.

5D. Critical evaluation and recommendation

Two limitations of a simple Bandit approach are: (1) it ignores the current traffic state and transition dynamics, so the same arm may be selected even when different approaches are congested; and (2) it optimizes immediate/statistical arm reward without explicitly planning long-term queue evolution, phase compatibility or fairness. It also needs safety constraints external to the learning algorithm.

Recommendation: use an MDP-based RL agent as the main controller because the traffic problem is sequential and state-dependent. A Bandit method is useful as a supporting experiment to compare candidate timing parameters or to initialize/tune green splits. A hybrid approach can use Bandit optimization to tune timing parameters while an MDP policy chooses safe state-dependent phases. In every case, hard safety constraints and a fallback controller are mandatory.

5E. Reproducibility and viva readiness

- Organize source code into Task01 through Task05 folders and include the input/configuration data.
- Include README.md with environment setup, Python/library versions, commands, seed values and brief task results.
- Capture genuine screenshots of terminal output and generated plots after running the scripts.
- Commit each task separately and share the repository link on the cover page and in the report.
- Be prepared to explain the MDP tuple, transition probabilities, reward design, Bellman calculation, ε-greedy exploration and one code modification.

Conclusion

The smart traffic signal problem is naturally modelled as a sequential decision-making task. The RL framework identifies the controller, intersection, state, actions and reward. The MDP formalizes state

transitions and discounted returns, while policies and Bellman equations provide a basis for evaluating and improving decisions. Bandit algorithms are useful for comparing independent timing options, but they do not capture the full state-transition structure of traffic. Therefore, an MDP-based RL controller, supported by Bandit-based timing experiments and enforced safety constraints, is the most appropriate design direction.

Executed experiment results (Task 5C)

The accompanying Task05/traffic_experiment.py script was executed with the fixed configuration (80 episodes × 12 decisions, seed 23) and the supplied illustrative traffic_arrivals.csv dataset. The following values are the actual output from that run, not assumed or fabricated results.

Controller	Average reward	Average mean queue	Arm selections [0,1,2,3]
Fixed-time	-133.1250	33.6875	[240, 240, 240, 240]
Greedy Bandit	-122.3253	31.0760	[16, 15, 3, 926]
ε-greedy Bandit	-135.1839	34.1716	[72, 836, 22, 30]

Under this particular simplified simulation, the greedy Bandit obtained the highest (least negative) average reward and the lowest average mean queue. It selected timing arm 3 for most decisions. The ε-greedy controller explored more and, in this finite run, did not outperform the greedy controller. This result is specific to the stated arrival dataset, service-capacity assumptions, reward equation and random seed; it does not establish that greedy control is generally superior or safe for real intersections. The Bandit comparison does not model full state-dependent traffic transitions.

Reproducibility: run `python Task05/traffic_experiment.py` from the supplementary package directory. The script writes `Outputs/traffic_experiment_summary.csv` and `Outputs/traffic_experiment_results.json`.